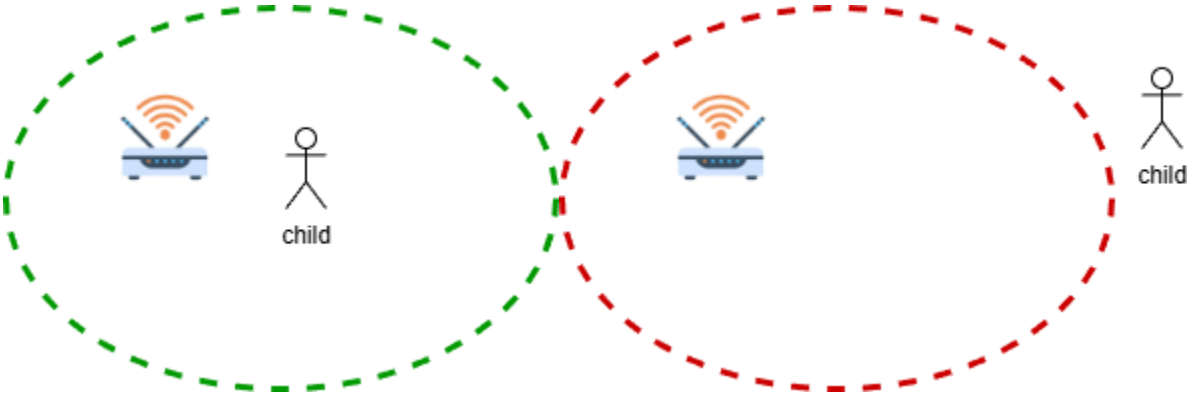
Contents

Introduction 2
Screen Mockups 3
Child Activity 3
Parent Activities 4
Use Case 6
Technical Description 7
Data persistence 7
Architecture Components 8
Hardware Features 8
UI Design 9
Foreground Service 9
Realtime Attendance Status 11
Attendance History 12
Conclusion 13
Potential Future Improvements 13
Testing 13

Introduction

ClassMate is a Kotlin mobile application designed to enhance the safety and monitoring of children's attendance at school. The system consists of:

- Child App: a background service running on the child's phone that automatically detects when the child enters or leaves the school area by analysing nearby Wi-Fi signals.
- Parent App: a complementary interface for parents that displays in real time when the child has entered or left the school, along with a comprehensive attendance history.



Screen Mockups

Child Activity

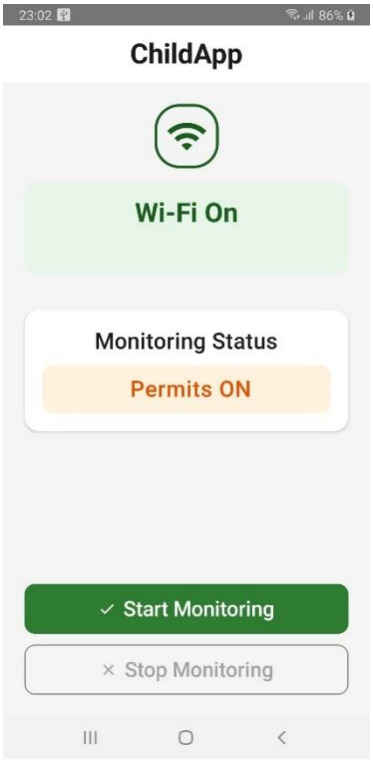


Image1

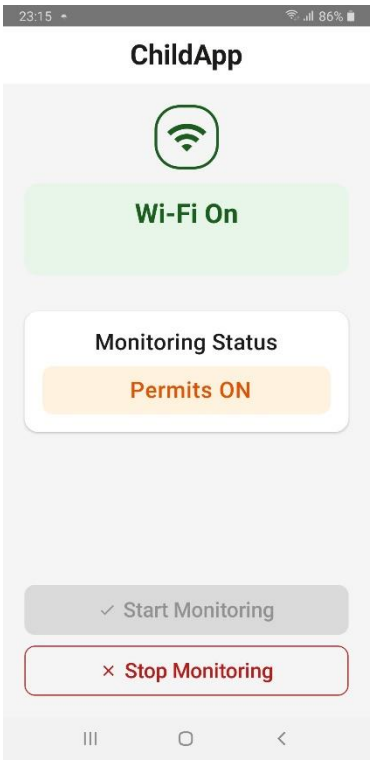


Image2

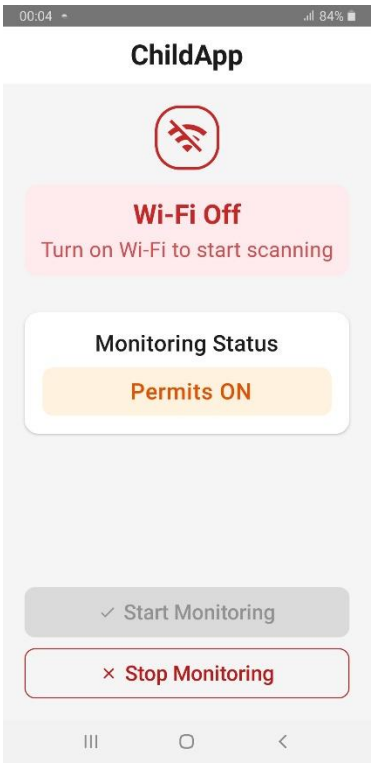


Image3

Image 1 Represents the initial state of the app when the device Wi-Fi is active, and the necessary permissions have been granted. The user sees the Wi-Fi status, permission checks, and can start monitoring by tapping the Start Monitoring button.

Image 2 The start button is disabled, and the Stop Monitoring option is displayed, allowing the user to stop the monitoring process at any time.

Image 3 The app clearly indicates that Wi-Fi is turned off and prompts the user to enable it in order to begin scanning and monitoring.

Parent Activities

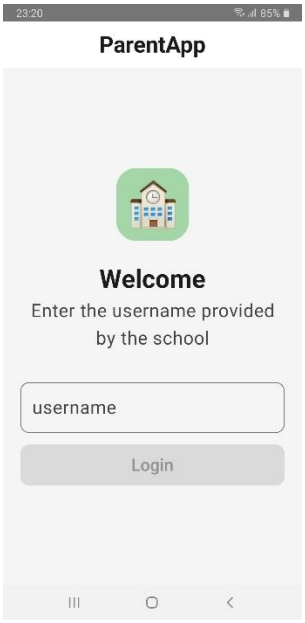


Image4

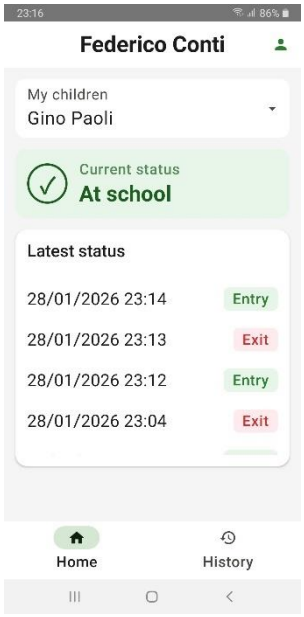


Image5

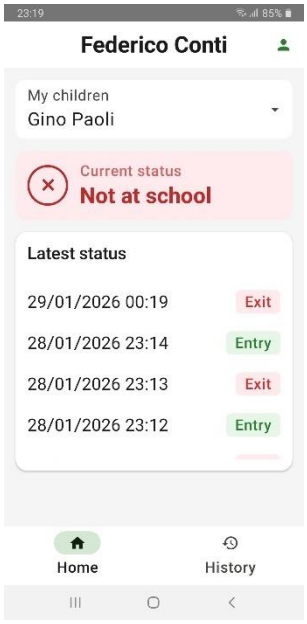


Image6

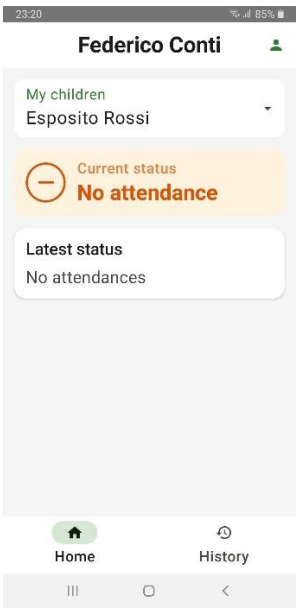


Image7

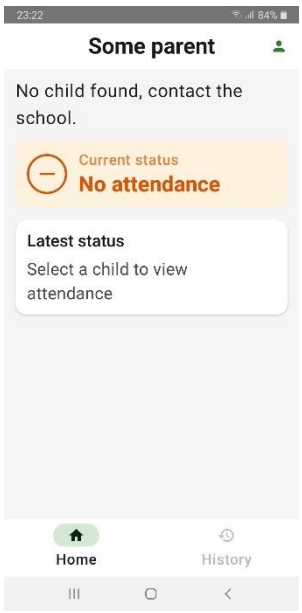


Image8

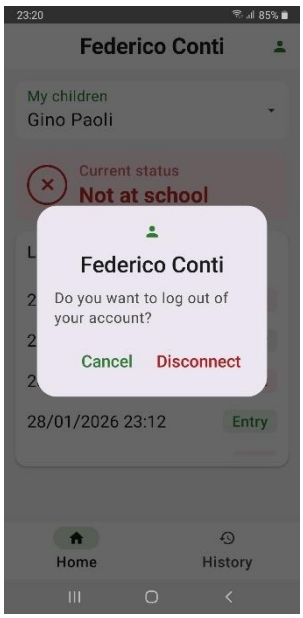


Image9

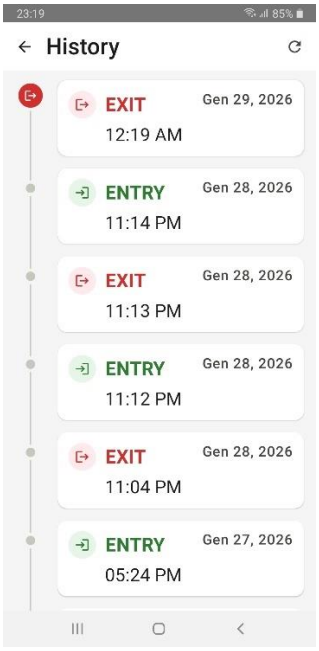


Image10

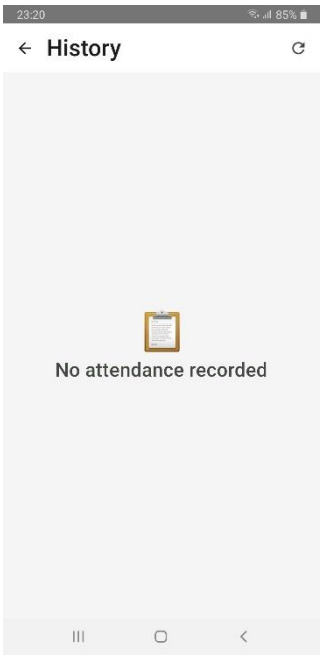


Image11

Image 4 The user enters the username provided by the school to authenticate.

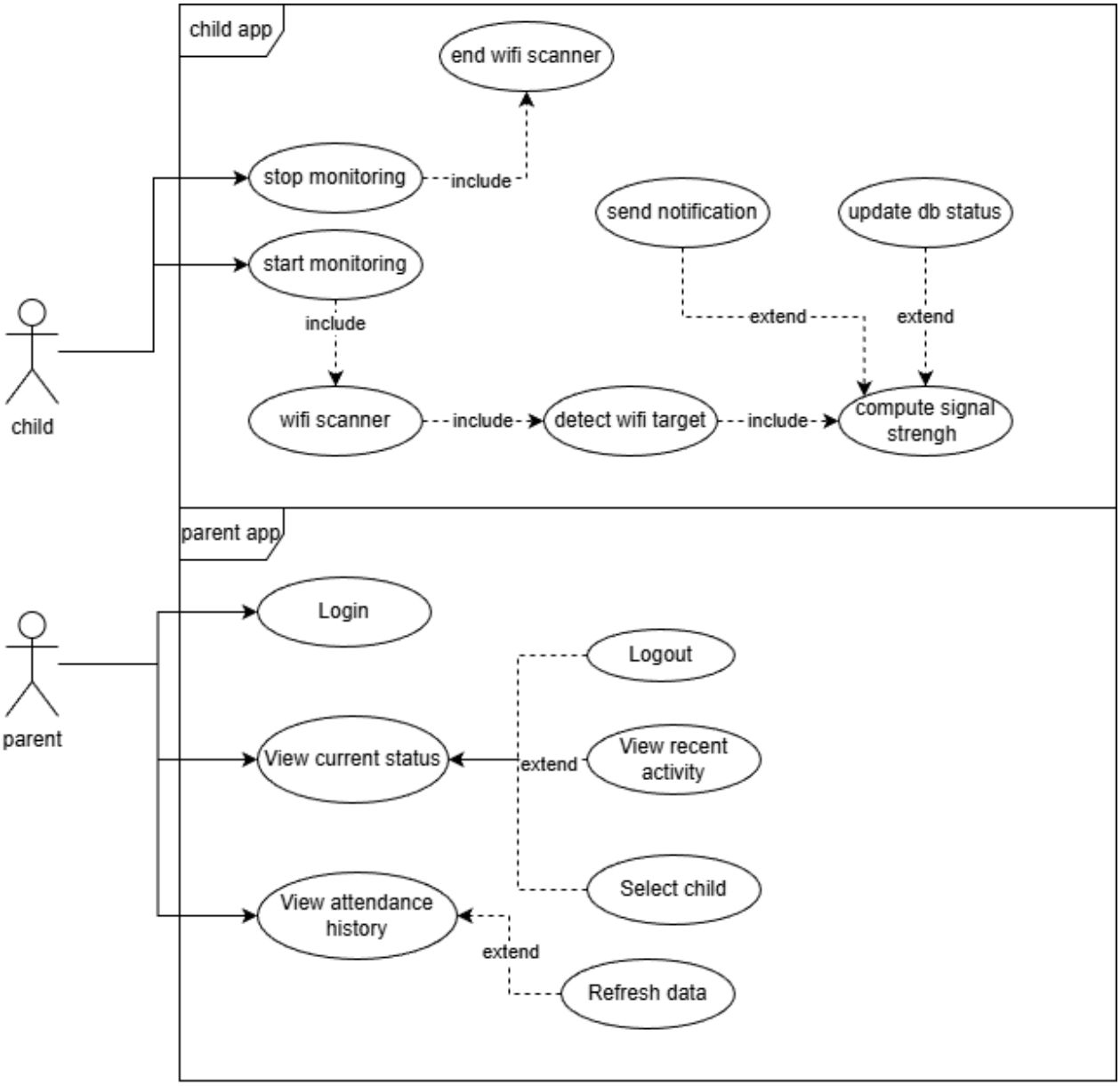
Images 5-6-7 These show the real-time attendance status of the selected child. The current status is displayed as "At/Not school," along with a list of the last 5 check-in and check-out events, including date and time. If the selected child has no recorded attendance yet, the app will notify the user. Another child can be selected using the dropdown menu.

Image 8 Informs the parent that no child was found associated with the account. The app suggests contacting the school to resolve the association issue.

Image 9 A modal dialog that allows the user to confirm or cancel logging out of the application.

Images 10-11 This Activity shows the complete attendance history, organized chronologically. When no attendance data is available for the selected student, the application displays a notification.

Use Case



Use Case Diagram illustrates the interactions between the two main actors: Child and Parent. The Child interacts indirectly through the app running in the background, which detects the Wi-Fi network, analyzes the signal strength, and automatically updates the database with entry or exit data. The Parent, on the other hand, uses the app with a graphical interface to login, view the child's status, consult attendance history, and select multiple children associated with their account.

Technical Description

Data persistence

The application utilizes a combination of local and cloud-based storage solutions for data persistence:

- **Local Storage**

DataStore is used for storing simple key-value data, specifically the parent's username for automatic login persistence. This allows the app to remember the logged-in parent across app sessions and device restarts. The UserPreferencesRepository class encapsulates all DataStore operations, providing a clean API for saving, retrieving, and clearing the stored parent ID.

- **Cloud Database**

Firebase Firestore is integrated as the primary database providing real-time data synchronization and offline support.

Firebase start structure:

```
{
  "parents": {
    "{parentId}": {
      "name": "string",
      "surname": "string",
      "childrens": "array"
    }
  },
  "childrens": {
    "{childId}": {
      "name": "string",
      "surname": "string",
      "attendance": {
        "{attendanceId}": {
          "type": "string",
          "time": "Timestamp"
        }
      }
    }
  }
}
```

Firebase is used in

- childApp.AttendanceRepository class manages attendance record creation
    - writes entry/exit events to Firestore
    - retrieves the most recent attendance type to prevent duplicates

- parentapp.MyAppRepository class handles all parent-side database operations
    - parent authentication and profile retrieval
    - children loading data with batch queries
    - real-time attendance monitoring a bidirectional callback channel register
    - paginated history retrieval with cursor-based pagination

Architecture Components

The application follows the Model-View-ViewModel (MVVM) architectural pattern, incorporating Android Architecture Components to ensure a robust, maintainable, and testable codebase.

- childapp.ChildViewModel class manage UI state for the child monitoring interface, handling permissions, service lifecycle, and Wi-Fi status
- parentapp.ParentViewMode classl orchestrates authentication, child selection, real-time attendance updates, and paginated history loading

Uses *StateFlow* and *MutableStateFlow* to expose immutable UI state to the view layer

Hardware Features

The ChildApp application leverages several native device capabilities to provide attendance tracking functionality:

- **Wi-Fi Scanning**

The app relies on the device's Wi-Fi hardware to perform continuous network scans, detecting nearby access points and measuring signal strength (RSSI). These scans enable proximity detection by identifying when the device is within range of a specific school network, identified by its SSID and its BSSID.

- **Push Notifications**

The notification system serves dual purposes: it maintains the mandatory persistent notification required for foreground services, and it provides real-time feedback to users about the app's operational status. Notifications display the current scanning state, attendance events (entry/exit), and any issues such as disabled Wi-Fi or missing permissions.

- **Permissions**

The application requires several runtimes and installs-time permissions to function correctly.

- Wireless Connectivity and Proximity Detection
- Contextual Awareness and Background Persistence

- User Communication and System Transparency

UI Design

The user interface of both applications follows the Material Design 3 guidelines, ensuring visual consistency, accessibility, and a modern user experience. The adopted design system includes:

- Color Scheme: A defined color palette to represent the application's states (presence/absence, service active/inactive, success/error)
- Typography: Consistent typographic hierarchy using MaterialTheme.typography
- Spacing System: A predefined sizing system (AppDimensions) for padding, margins, and element dimensions
- Shape System: Rounded borders and consistent shapes through MaterialTheme.shapes
- Elevation: Use of shadows and layers to create visual depth in cards and components

Foreground Service

ChildApp starts a Foreground Service that continues to run even when the application is closed or terminated. The service continuously monitors the Wi-Fi signal of a specific target network and automatically records "entry" and "exit" events in the Firebase database when it detects stable changes in the user's proximity to the network.

- **Service Startup**

The service is started by MainActivity via an Intent containing the necessary parameters:

- *EXTRA_CHILD_ID: unique identifier for the child (e.g., "s4991779")*
- *EXTRA_CHILD_NAME and EXTRA_CHILD_SURNAME: name and surname*
- *EXTRA_TARGET_SSID: name of the Wi-Fi network to monitor (e.g., "badc0ff3")*
- *EXTRA_TARGET_BSSID: MAC address of the router (e.g., "6e:ed:a1:91:fc:f5")*

- **Service Initialization**

At startup, the ForegroundWifiScanService class:

- Creates persistent notification to keep the service running in the foreground
- Retrieves the last record type from the database (*getLastType*) to avoid duplicates
- Registers the *WifiScanReceiver::BroadcastReceiver* to receive Wi-Fi scan results
- Starts a continuous scan loop every 2 minutes **

** Starting from Android 9, each foreground app can scan four times in a 2-minute period.

- **Wi-Fi Scanning**

The WifiScanReceiver class filters Wi-Fi scan results by searching for the target network:

- Compares the SSID of the network with the specified one
- If present, also checks the BSSID for greater precision
- When the target network is found, it passes the signal strength (RSSI in dBm) to the WifiScanProcessor class

- **Signal Processing**

The Wi-Fi signal fluctuates continuously even if the person is not moving. If the service reacted to every small variation, it would generate false state changes.

The WifiScanProcessor class implements a filtering system to avoid false positives.

Solution:

- Weighted Average on Moving Window + Stability Filter

```
STATE VARIABLES:
    samples: List<Integer> = []          // RSSI samples buffer
    currentState: State = OUT            // current proximity state
    previousState: State = OUT           // previous proximity state
    stableCount: Integer = 0             // stability counter

CONSTANTS:
    WINDOW_SIZE = 10
    THRESHOLD_DBM = -45
    STABLE_DECISIONS_NEEDED = 3
    STATES = {IN, OUT}
```

```
FUNCTION onNewRssiSample(levelDbm: Integer) -> String OR NULL:

    // Step 1: Add new sample to the window
    samples.append(levelDbm)

    // Step 2: Wait until we have enough samples
    IF samples.length < windowSize THEN
        RETURN NULL  // Not enough data yet
    END IF

    // Step 3: Calculate weighted average of samples
    avg = weightedAverage(samples)

    // Step 4: Update state based on threshold
    previous = state
    IF avg > thresholdDbm THEN
        state = IN   // Signal strong enough: device is near
    ELSE
        state = OUT  // Signal too weak: device is far
    END IF
```

```
    // Step 5: Update stability counter
    IF state != previous THEN
        stableCount = 0  // State changed, reset counter
    ELSE
        stableCount = stableCount + 1  // State unchanged, increment
    END IF

    // Step 6: Remove oldest samples (sliding window)
    // Remove 1/3 of window to make room for new samples
    removeCount = windowSize / 3
    FOR i = 1 TO removeCount DO
        IF samples is not empty THEN
            samples.removeFirst()
        END IF
    END FOR

    // Step 7: Check if we have stable decision
    IF stableCount == stableDecisionsNeeded THEN
        stableCount = 0  // Reset counter for next event

        IF state == IN THEN
            RETURN "entry"  // Confirmed entry event
        ELSE
            RETURN "exit"   // Confirmed exit event
        END IF
    END IF

    // Step 8: Not stable yet
    RETURN NULL  // No event to report

END FUNCTION
```

The weighted average assigns greater weights to the most recent samples (at the end of the list) and lesser weights to the oldest samples (at the beginning of the list).

- **Database Registration**

When a stable event is detected, the service:

- Checks for duplicates: it compares the new type (entry/exit) with the last record saved in the collection.
- If different, it adds a new document to the Firestore collection.

This control mechanism ensures:

- Prevents duplicates: it does not record two consecutive entries or exits.
- Ensures consistency: the sequence of records always alternates "Entry → Exit → Entry..."

Realtime Attendance Status

ParentApp provides real-time updates on the selected child's attendance status, displaying the last 5 attendance records and determining whether the child is currently at school or not.

The functionality is based on three main components:

- **Repository Layer**

The ParentRepository exposes a *listenLastAttendance* method that creates a real-time listener on Firestore:

- *callbackFlow* creates a bidirectional communication channel between Firestore and the app
- *addSnapshotListener* registers a listener that automatically receives updates whenever data changes in Firestore
- Each database modification generates a new snapshot that is sent through the Flow

- **ViewModel Layer**

The ParentViewModel exposes two StateFlows that react to changes in the selected child:

- last5Records: The last 5 records
- childPresenceStatus: Current attendance status

- **UI Layer**

The HomeScreen observes both states and renders them using *StatusIndicator* and *LastRecordsTable* components.

Attendance History

The History Screen displays a chronologically ordered timeline of all attendance records for a selected child, implementing infinite scroll pagination to efficiently load large datasets.

The functionality is based on three main components:

- **Repository Layer**

The ParentRepository exposes a *getAttendancePage* method that

- Queries Firestore's attendance subcollection for the specified child
- Orders record by time in descending timeline order
- Limits results to page size (default: 20 records)
- Uses startAfter(cursor) to fetch the next page when a cursor is provided
- Returns both the records and a cursor pointing to the last document, enabling subsequent page loads
- Sets nextCursor to null when fewer records than pageSize are returned, indicating the end of the data

- **ViewModel Layer**

The ParentViewModel manages pagination state through

- _historyUiState: StateFlow<HistoryState> that holds the current UI state (items list, loading flags, error messages) historyCursor: A private DocumentSnapshot? that tracks the Firestore cursor for pagination continuity

- **UI Layer**

The HomeScreen observes the ViewModel state and triggers pagination actions:

- When user open HistoryScreen and When the user scrolls near the bottom of the list
    - Update state with the first page of records and the new cursor
    - Monitors scroll position using LazyListState
    - Check if user reaches within 5 items of the bottom
    - Appends new records to existing items (filtering duplicates by ID)
    - Renders all items in a LazyColumn component

- When user the user pulls to refresh into HistoryScreen
    - Update state with the first page of records and the new cursor

Conclusion

The developed application allows for automatic monitoring of a child's presence in a specific location (such as a school) by periodically scanning available Wi-Fi networks. The implementation uses a foreground service for Wi-Fi scanning, with filtering logic based on RSSI and identifiers of the target networks (SSID/BSSID) and integrates Firebase Firestore as the backend for recording attendance.

Potential Future Improvements

Several potential improvements could significantly improve the system's security, reliability, and functionality:

- SSO integration with external providers (Google, Apple, Microsoft)
- Notify parents immediately when their child enters or exits the school premises using Firebase Cloud Messaging (FCM
- When the device loses connectivity, implement a timeout mechanism to automatically register an "exit" event when the target MAC address is no longer detected for a specified duration:
- When the target MAC address is not detected, implement a timeout mechanism to automatically register an "exit" event.
- Leverage LLMs API to generate insights and statistics from historical attendance data.
- RAG Chatbot to query the knowledge base of historical attendance records

Testing

The application was primarily tested on Android 10. Due to Android limitations, to enable reliable testing, Wi-Fi scan throttling was disabled using the following developer option:

Since Android 8.0, Wi-Fi scanning is considered a form of indirect location; therefore, location permission is required.

Furthermore, in the Android settings, there is an option called "Wi-Fi scanning" (Settings > Location > Wi-Fi scanning) which, if enabled, allows scanning even when Wi-Fi is turned off. Despite this, the app explicitly requires Wi-Fi to be active: if Wi-Fi is turned off, the scanning function (scanLoop) does not start, to ensure consistency and reliability in data collection.

To test the application, you must configure the required parameters (e.g., childId, SSID, BSSID) in the strings.xml file located in the Resource directory (res/values/strings.xml).